

Custom Password Generator

Console-Based Secure Password Generator built in Python

Intern	Nimra	Organization	Decode Labs
Program	Python Development Internship	Language	Python 3.12.4
Document Type	Project Documentation Report	Status	Completed

1. Project Title & Overview

The Custom Password Generator is a lightweight, console-based Python utility that produces cryptographically strong, random passwords of a user-specified length. It uses Python's built-in **secrets** module — rather than the standard **random** module — to ensure the generated output is suitable for security-sensitive use cases such as account passwords and access keys. The program combines uppercase and lowercase letters, digits, and punctuation symbols to maximize password entropy, while gracefully validating user input to prevent runtime crashes.

2. Objectives & Features

Objectives

- Generate strong, unpredictable passwords using a cryptographically secure random source.
- Allow the user to define a custom password length via command-line input.
- Ensure the tool is robust against invalid or non-numeric input.
- Demonstrate practical use of Python's **secrets** and **string** standard library modules.

Key Features

- Combines letters (upper & lower case), digits, and punctuation for high entropy passwords.
- User-defined password length via interactive CLI prompt.
- Input validation with a minimum length check (length must be at least 1).
- Exception handling (**ValueError**) for non-integer input, preventing program crashes.
- Clean, single-responsibility **generate_password()** function that can be reused or imported.

3. System Requirements & Technologies Used

Item	Details
Language	Python 3.12.4
Standard Modules Used	secrets — cryptographically secure random selection string — character set constants (letters, digits, punctuation)
External Dependencies	None (uses only the Python Standard Library)

Operating System	Windows / macOS / Linux (cross-platform)
Execution Environment	Any terminal / command prompt with Python 3 installed
Entry File	main.py

4. Code Architecture & Workflow

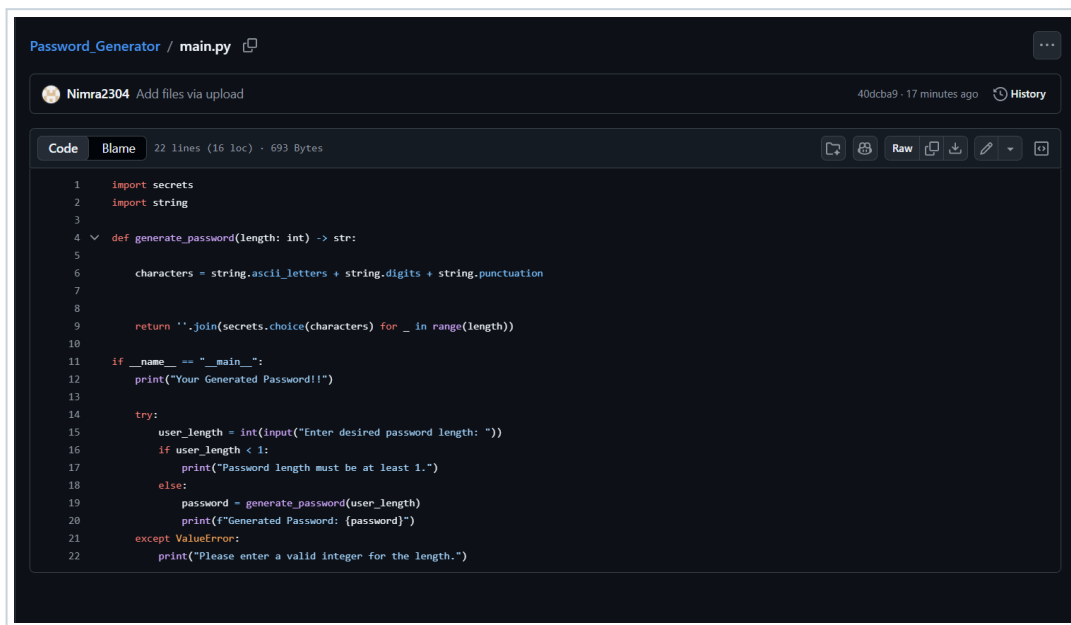
The program is structured around a single utility function, `generate_password(length: int) -> str`, which builds a character pool from `string.ascii_letters`, `string.digits`, and `string.punctuation`, then uses `secrets.choice()` inside a list comprehension to randomly select characters for the requested length. Using `secrets` instead of `random` is a deliberate security decision, since `secrets` is designed for generating tokens, passwords, and other values that must remain unpredictable.

Program execution begins under the `if __name__ == "__main__":` guard, which prompts the user for a desired password length via `input()`. This input is wrapped in a `try/except` block that catches `ValueError` exceptions, ensuring that non-numeric input (e.g. letters or blank input) does not crash the program. A secondary validation check rejects lengths less than 1 with a clear error message. Once validated, the length is passed to `generate_password()` and the resulting password is printed to the console using an f-string.

5. How to Run the Program (CLI)

- 1 Ensure Python 3.12.4 (or a compatible Python 3 release) is installed by running `python --version` (or `python3 --version`) in your terminal.
- 2 Clone or download the repository: `git clone https://github.com/Nimra2304/Password_Generator.git`
- 3 Navigate into the project directory: `cd Password_Generator`
- 4 Run the script: `python main.py` (use `python3 main.py` on macOS/Linux).
- 5 When prompted with **Enter desired password length:**, type a positive integer (e.g. 16) and press Enter.
- 6 The generated password is printed to the console as **Generated Password: <output>**.

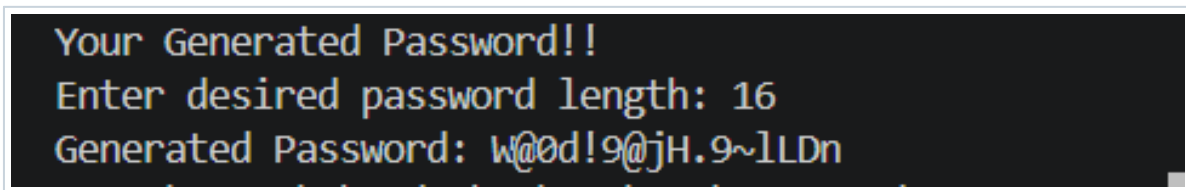
6. Screenshots



The screenshot shows the GitHub repository view for the file `main.py` in the `Password_Generator` repository. The code is displayed in a dark-themed editor with line numbers 1 through 22. The code imports `secrets` and `string`, defines a `generate_password` function, and includes a `__main__` block for user interaction.

```
1 import secrets
2 import string
3
4 def generate_password(length: int) -> str:
5
6     characters = string.ascii_letters + string.digits + string.punctuation
7
8     return ''.join(secrets.choice(characters) for _ in range(length))
9
10
11 if __name__ == "__main__":
12     print("Your Generated Password!!")
13
14     try:
15         user_length = int(input("Enter desired password length: "))
16         if user_length < 1:
17             print("Password length must be at least 1.")
18         else:
19             password = generate_password(user_length)
20             print(f"Generated Password: {password}")
21     except ValueError:
22         print("Please enter a valid integer for the length.")
```

Figure 1 — `main.py` source code (GitHub repository view).



The screenshot shows a terminal window with the following output:

```
Your Generated Password!!
Enter desired password length: 16
Generated Password: W@0d!9@jH.9~1LDn
```

Figure 2 — Sample terminal execution: password length input and generated output.

7. GitHub Repository Link

https://github.com/Nimra2304/Password_Generator/tree/main